BiasClean Toolkit

Phase 4: Internal Production Hardening

Dependency Pinning, Containerization, Structured Logging, and Graceful Error Handling

Author: Hamid Tavakoli (h.tavakoli@ai-fairness.com) | Repository: github.com/AI-Fairness-com/BiasClean

Pipeline version at completion: 3.10.4

1. Purpose and Scope

Phases 1 through 3 hardened and validated what BiasClean's pipeline decides about a disparity: its detection logic, its rebalancing behavior, its safeguards against overcorrection, and its performance against an independent benchmark (AIF360, judged by Aequitas). Phase 4 changes none of that decision-making. It wraps the now-settled fairness logic in the engineering practices needed to run it reliably, reproducibly, and diagnosable over time, in an internal-use-only context (no authentication, no multi-tenancy, no external-facing deployment concerns).

Phase 4 was scoped into four workstreams, lettered D through G to continue the project's existing changelog sequence without renumbering anything already shipped:

- Workstream D, Dependency pinning (requirements.txt, exact versions)
- Workstream E, Dockerfile (reproducible environment)
- Workstream F, Structured logging (replacing print() with real logging)
- Workstream G, Graceful malformed-input handling

All four are complete, documented in docs/CHANGELOG.md (versions 3.10.1 through 3.10.4), and validated against the project's five established real-world datasets: UCI Communities & Crime, ProPublica COMPAS, Oklahoma City traffic stops, the NIJ Recidivism Challenge, and North Carolina statewide traffic stops (20.3 million rows).

2. Motivating Incidents

Phase 4 was not undertaken speculatively. Five concrete incidents from this project's own history motivated it directly:

- An aequitas==1.0.0 upgrade pulled in fairgbm, which ships a precompiled binary incompatible with Apple Silicon and crashed outright. Reverting to 0.42.0 fixed it, but nothing in the project pinned this before it happened.
- Installing xhtml2pdf==0.2.17 (needed for BiasClean's own PDF reports) triggered a pip resolver warning against aequitas 0.42.0's own declared xhtml2pdf==0.2.2 requirement, the same category of risk, caught before it caused a real failure.
- Repeated terminal paste/heredoc failures (nano opening and closing with 0 bytes written) were a recurring friction point in this project's own workflow that a more reproducible environment would reduce.
- Every local run's console output was print() statements, not filterable by severity, not timestamped, not automatically captured to a file.
- Several datasets (NIJ, North Carolina) needed ad hoc, dataset-specific type-coercion guards written outside the pipeline, in aif360_benchmark_native.py, rather than the pipeline handling the underlying assumption natively, a sign worth auditing properly rather than patching around indefinitely.

3. Workstream D, Dependency Pinning

requirements.txt previously used permissive >= version bounds. Converted to exact == pins for all 17 real pipeline runtime dependencies (pandas, numpy, scipy, matplotlib, seaborn, scikit-learn, fairlearn, reportlab, xhtml2pdf, python-bidi, pypdf, Pillow, html5lib, pyHanko, pyhanko-certvalidator, arabic-reshaper, svglib), using the versions already confirmed working. aequitas and aif360, used only for this project's own benchmarking, never a pipeline runtime dependency, were split into a separate requirements-dev.txt, along with the documented aequitas/xhtml2pdf version tension.

Validation

A fresh install into a clean virtual environment from the pinned file resolved every package to its exact intended version with zero resolver errors. This validation pass itself caught real drift the old bounds would have caused: reportlab would have silently resolved to an untested 5.0.0 (instead of the confirmed-working 4.5.1), and svglib to an untested 2.1.0 (instead of 2.0.2). A subsequent Communities & Crime re-run in the clean environment reproduced the on-record bias score exactly (0.0734 → 0.0268).

Shipped as v3.10.1.

4. Workstream E, Dockerfile

A Dockerfile was built pinning python:3.9.6-slim (matching the project's own confirmed working Python version), copying requirements.txt as an early, cacheable layer, and deliberately not baking datasets into the image. The North Carolina file alone is 4.87GB, so real datasets are mounted as Docker volumes at runtime instead. The pipeline's existing interactive terminal prompt was preserved as the entry-point; a non-interactive CLI mode was deliberately not added, as a scope decision for this workstream.

The North Carolina Memory Challenge

North Carolina's 20,286,645-row file required Docker Desktop's memory allocation to be raised well above its 8GB default. The raw DataFrame alone occupies approximately 22GB in memory before any rebalancing step begins. Two attempts failed with exit code 137 (Docker's out-of-memory signal), first at 8GB, then at 24GB (the second attempt also mistakenly left SVM fairness enforcement enabled, which adds further memory burden). Raising the limit to approximately 30GB (on a 36GB-RAM machine) succeeded, with actual peak usage observed at 26–29GB. SVM fairness enforcement must stay disabled for North Carolina regardless of environment, a constraint documented since Phase 1/2, now confirmed to hold in Docker as well as natively.

Validation

All 5 established datasets were run inside the built container and reproduced their on-record bias scores exactly, including North Carolina's three documented group reversals (Ethnicity, Region, Age) and its Gender worst-group regression finding.

A companion docs/DOCKER_SETUP.md was written documenting build/run commands, the memory-tuning guidance, the SVM-disabled-for-NC constraint, and a GitHub web-upload gotcha encountered along the way (the leading dot was silently stripped from .dockerignore during upload, requiring a rename via GitHub's file editor to fix).

Shipped as v3.10.2.

5. Workstream F, Structured Logging

261 print() calls inside the pipeline's own diagnostic output, feature mapping, constraint validation, rebalancing, SVM enforcement, safeguards, gates, and reporting, were converted to Python's logging module, mapped to severity by message content rather than mechanically: phase progress banners to INFO (231 calls), safeguard/gate/reversal/mapping-tie warnings to WARNING (28 calls), and the pipeline's own top-level failure handler to ERROR (2 calls). No message's wording was changed anywhere, only how and where each is emitted. The interactive CLI's menus and prompts, and the sample-dataset generators, were deliberately left untouched as direct user-interface text rather than diagnostic output.

A fresh, timestamped log file (biasclean_results/run_<timestamp>.log) is now attached automatically at the start of every pipeline run, alongside audit_trail.json and report.pdf. One implementation detail worth recording: Python's default logging handler writes to stderr, while print() writes to stdout; left unaddressed, this would have silently broken anyone piping the pipeline's console output to a file. The console handler was explicitly configured to target stdout to preserve identical behavior.

Validation

Verified via direct functional testing (a full pipeline run executed twice in the same process, confirming clean handler hygiene with no accumulation or duplication across runs) before reaching the project's own machine. Further validated on real data: COMPAS and Communities & Crime both reproduced their exact on-record bias scores in native venv and in Docker, four runs total, all exact matches, with log files correctly capturing WARNING-level entries (tied-outcome-candidate ambiguity, validation-gate rejections, mapping ties) alongside routine INFO progress.

Shipped as v3.10.3.

6. Workstream G, Graceful Malformed-Input Handling

This workstream called for an audit of the pipeline's dtype and data-shape assumptions, rather than a fix for one reported incident, and the audit surfaced two real, previously undiscovered issues.

Finding 1: Fully-Null Protected Attribute Columns

A protected attribute column that is entirely missing (no non-missing values overlapping with the target) previously passed validation with only a generic warning rather than a blocking error. The root cause: calling .min() on the value-count of an empty post-dropna Series silently returns NaN rather than raising, and the comparison NaN < 50 evaluates to False, so the pipeline's own small-group-size warning never fired. Because auto-approval blocks only on validation errors, never warnings, a column with zero usable data could be silently auto-approved and produce a silent no-op during rebalancing, with nothing indicating the analysis had measured nothing at all.

Fix: An explicit check now raises a clear, specific error naming the column and target the moment zero overlapping non-missing rows are detected, before any of the logic that previously masked the problem runs.

Finding 2: Silent Substitution on a Missing Target Column

The more significant finding. When a user specifies a target column name that does not exist in the dataset, a typo, or a column dropped during upstream cleanup, the pipeline was silently falling through to auto-detection and selecting a different column instead, without any indication that the requested column was ever missing. This directly contradicted the surrounding code's own documented intent, stated in an adjacent comment: "no silent substitution when someone has told us exactly what they want." The condition governing this logic checked only for "specified and present"; there was no corresponding branch for "specified but not found" and it fell through to auto-detection by default. A user with a typo'd column name would receive a complete-looking report auditing bias against a column they never asked for.

Fix: The pipeline now fails immediately with a clear error naming the missing column and previewing the dataset's actual columns, rather than silently substituting a different one.

Validation

Both fixes were validated against four synthetic test cases: a fully-null protected column (now correctly rejected), a nonexistent target column (now correctly raises a clear error instead of silently substituting), a target column with 3+ classes (confirmed pre-existing correct behavior, unaffected), and normal well-formed data (confirmed to run end-to-end identically, with zero behavioral change). Both fixes were further confirmed correctly inert on real data: COMPAS and Communities & Crime both reproduced their exact on-record bias scores in Docker after the fixes were applied.

Shipped as v3.10.4.

7. Cross-Environment Validation Summary

Dataset	Bias Score (Initial → Final)	Environments Validated	Notable Findings
Communities & Crime	0.0734 → 0.0268	Native venv, Docker (×3 builds)	Consistent baseline across all workstreams
COMPAS	0.0949 → 0.0521	Native venv, Docker (×3 builds)	SVM gate correctly rejects Ethnicity reversal each run
Oklahoma City	0.0003 (unchanged)	Docker	Near-null signal, correctly reported as such
NIJ	0.0468 (unchanged)	Docker	Correctly below mitigation threshold
North Carolina	0.0670 → 0.0314	Docker (memory-tuned)	20.3M rows; required raising Docker's memory limit to ~30GB

8. Conclusion

Phase 4 closes out the project's own stated Definition of Done for internal production hardening: pinned, reproducible dependencies; a working, memory-tuned containerized environment validated even at North Carolina's 20-million-row scale; real, severity-filterable diagnostic logging in place of bare console output; and a genuine audit of malformed-input handling that surfaced and fixed two real defects, one of them, the silent target-column substitution, a legitimate trust-affecting bug rather than routine hardening. Every change was validated against the project's own on-record results before being considered complete, consistent with the root-cause-first, verify-against-real-data standard this project has held itself to since Phase 1.

With Phase 4 complete, BiasClean's pipeline (v3.10.4) has now been hardened across four full phases: consolidation and regression testing (Phase 1), external validation against six real-world datasets (Phase 2), an independent blind benchmark against AIF360 judged by Aequitas (Phase 3), and internal production hardening (Phase 4), a complete, evidence-backed record from first bug fix to containerized, logged, defensively-validated production tool.